

HPCG2 - Reto 1

Solucion de la ecuacion de Poisson 1D con Jacobi Iterativo (Serial, Threads y Procesos)

Integrantes

- David Santiago Lugo Cabrera
 - Jose Aroudo Junior de Asis Pinedo
 - Angie Carolina Vargas Villegas
-

1. Analisis del algoritmo de Jacobi Iterativo para Poisson 1D

Se considera el problema de frontera:

- $-u''(x) = f(x), x \text{ in } (0,1)$
- $u(0)=0, u(1)=0$

Con discretizacion por diferencias finitas de segundo orden sobre N nodos internos:

- $h = 1/(N+1)$
- $-u_{i-1} + 2u_i - u_{i+1} = h^2 f_i, i=1..N$

El metodo de Jacobi obtiene una sucesion $u^{(k)}$:

$$u_i^{(k+1)} = 0.5 * (u_{i-1}^{(k)} + u_{i+1}^{(k)} + h^2 f_i)$$

1.1 Costo computacional

- Costo por iteracion: $O(N)$.
- Costo total: $O(N * \kappa)$ donde κ depende de tolerancia y condicionamiento.
- Memoria: $O(N)$ usando dos vectores (u y u_{next}).

1.2 Criterio de convergencia

Se usa norma infinito sobre el cambio iterativo:

$$residual = \max_i |u_i^{(k+1)} - u_i^{(k)}|$$

El algoritmo termina cuando $residual < tol$ o al llegar a max_iter .

1.3 Dependencias de datos

- Cada $u_i^{(k+1)}$ depende solo de vecinos en iteracion previa (k).
- Dentro de la misma iteracion no hay dependencia de escritura entre nodos internos.
- Esto habilita paralelismo por particion de dominio (bloques de indices).

2. Implementacion serial

Se implemento en C en `src/serial/jacobi_serial.c` con:

- Vectores dinamicos para u , u_{next} y f .
- Intercambio de punteros por iteracion para evitar copia completa.
- Medicion de tiempo con `clock_gettime(CLOCK_MONOTONIC)`.

Resumen de flujo:

1. Inicializar $u=0$ y f .
2. Repetir actualizacion Jacobi sobre nodos $1..N$.
3. Calcular `residual` maximo de la iteracion.
4. Intercambiar $u \leftrightarrow u_{next}$.
5. Verificar convergencia.

3. Analisis de concurrencia y optimizacion (Threads y fork)

3.1 Threads (pthread)

Archivo: `src/threads/jacobi_threads.c`

Estrategia:

- Particion de nodos en τ bloques contiguos.
- Cada hilo actualiza su rango y calcula un maximo local.
- Barrera 1: esperar fin de calculo por iteracion.
- Hilo 0 reduce maximos, decide convergencia e intercambia punteros.
- Barrera 2: publicar estado para la siguiente iteracion.

Ventajas:

- Menor sobrecarga de comunicacion que procesos.
- Memoria compartida natural.
- Mejor latencia de sincronizacion para granularidades finas.

Riesgos:

- Sobresincronizacion si N es pequeno.
- Falsa comparticion en estructuras compartidas.
- Escalamiento limitado por ancho de banda de memoria.

3.2 Procesos (fork + memoria compartida)

Archivo: `src/processes/jacobi_processes.c`

Estrategia:

- `fork` de procesos trabajadores persistentes.
- Vectores alojados en memoria compartida (`mmap MAP_SHARED`).
- Barreras `pthread_barrier` con atributo `PTHREAD_PROCESS_SHARED`.
- El proceso padre reduce maximos locales, intercambia punteros y decide parada.

Ventajas:

- Aislamiento de espacio de direcciones.
- Modelo valido para escenarios de multiproceso y afinidad por CPU.

Riesgos:

- Overhead de sincronizacion y gestion de procesos mayor que en hilos.
- Mayor sensibilidad a costos de IPC/sincronizacion.
- Complejidad de implementacion y depuracion superior.

3.3 Optimizacion de CPU y memoria

Optimizaciones aplicables:

- Compilacion con `-O3`.
- Particion balanceada para evitar desbalance de carga.
- Evitar copias completas del vector (swap de punteros).

- Acceso secuencial a memoria para mejorar localidad espacial.
- Fijar afinidad CPU (opcional) para reducir migraciones de hilos/procesos.
- Ajustar numero de workers a nucleos fisicos disponibles.

4. Analisis de causas de desempeno

4.1 Comportamiento esperado

- Serial: referencia base.
- Threads: mejor speedup en tamanos de problema medios/grandes.
- Procesos: mejora posible, pero usualmente menor que threads por overhead.

4.2 Causas comunes de degradacion

- Costo de barreras por iteracion (especialmente con N bajo).
- Limite de ancho de banda de memoria en muchos workers.
- Desbalance por particiones no uniformes.
- Competencia de cache y falsa comparticion.
- Variabilidad del sistema operativo (scheduler, carga externa).

4.3 Metodologia de medicion ejecutada

1. Se uso el mismo problema para todas las pruebas: $N=200000$, $\text{max_iter}=2000$, $\text{tol}=1e-20$.
2. Se ejecutaron 10 corridas por configuracion.
3. Se midio `Tiempo_s` reportado por cada binario.
4. Se calculo promedio, desviacion estandar y mejor tiempo.
5. Se calculo speedup: $T_{\text{serial}} / T_{\text{paralelo}}$ y eficiencia: $\text{speedup} / \text{workers}$.

Entorno de prueba:

- CPU: AMD Ryzen 5 5600X (6 nucleos fisicos, 12 logicos).
- SO: Windows.
- Compilacion: -O3.

4.4 Tabla de resultados

Metodo	Workers	N	Tiempo promedio (s)	Desv. estandar (s)	Mejor tiempo (s)	Speedup	Eficiencia
Serial	1	200000	0.340550	0.021201	0.320326	1.0000	1.00
Threads	2	200000	0.252844	0.017542	0.241828	1.3469	0.67
Threads	4	200000	0.212823	0.017482	0.190592	1.6002	0.40
Threads	8	200000	0.202809	0.011062	0.191865	1.6792	0.20
Procesos	2	200000	N/D (Windows)	N/D	N/D	N/D	N
Procesos	4	200000	N/D (Windows)	N/D	N/D	N/D	N
Procesos	8	200000	N/D (Windows)	N/D	N/D	N/D	N

Nota: la version `fork` no se pudo ejecutar en Windows nativo por dependencia de APIs POSIX (`mmap`, `fork`, barreras de proceso compartido). Para medir procesos, se requiere Linux o WSL con distribucion instalada.

5. Conclusiones

- Las pruebas confirman que Jacobi 1D es paralelizable por particion de dominio, ya que

cada punto interno depende solo de vecinos de la iteracion previa.

- La version con `pthread` mejora el tiempo frente a serial, con speedup de $1.35\times$ (2 hilos), $1.60\times$ (4 hilos) y $1.68\times$ (8 hilos).
- La eficiencia cae al aumentar workers ($0.67 \rightarrow 0.40 \rightarrow 0.21$), lo cual evidencia overhead de sincronizacion por barrera y limite de ancho de banda de memoria.
- La ganancia de 8 hilos frente a 4 hilos es moderada, por lo que para este tamaño de problema una configuracion intermedia puede ofrecer mejor compromiso rendimiento/uso de CPU.
- La evaluacion de procesos queda pendiente para entorno Linux/WSL; aun así, por teoria y costos de IPC, se espera que `fork` tenga mayor overhead que `pthread` para granularidad fina por iteracion.